DSA STUDY BLUEPRINT SERIES

113. Course Schedule Problem Graph Topological Sort

Validating Course Order Dependencies (Course Schedule I)

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Concept:** Tasks with prerequisite constraints.
- Check if dependencies form a cycle. True if DAG, false if cyclic.

Memory & Execution Blueprint

Course 1 → Course 2 → Course 1 (Deadlock cycle!) ⇒ Schedule Impossible

C++ Implementation Reference

Standard code layout and syntax

```
// Construct directed graph; execute cycle detection using DFS path tracker.
// If cycle exists, dependencies cannot be met.
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(V + E)
Space Complexity	O(V + E)

Key Practices & Gotchas

- **Important:** Can also be solved using Kahn's topological sort.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.